CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

B. TECH (IT/CE/CSE/AIML)

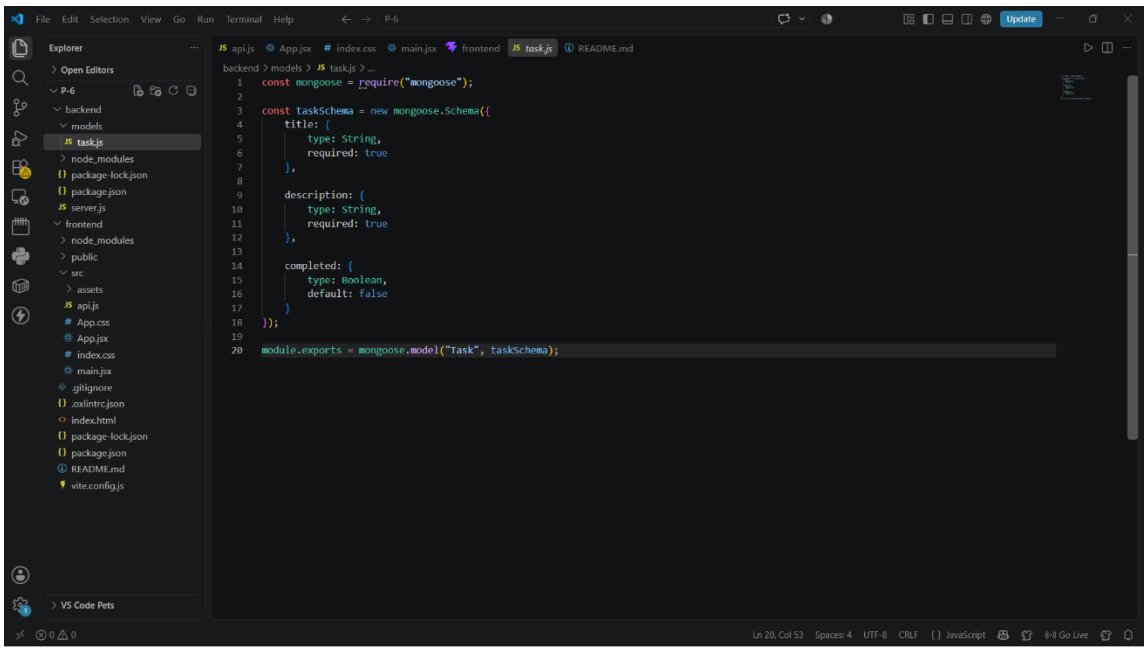
Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical Statement:

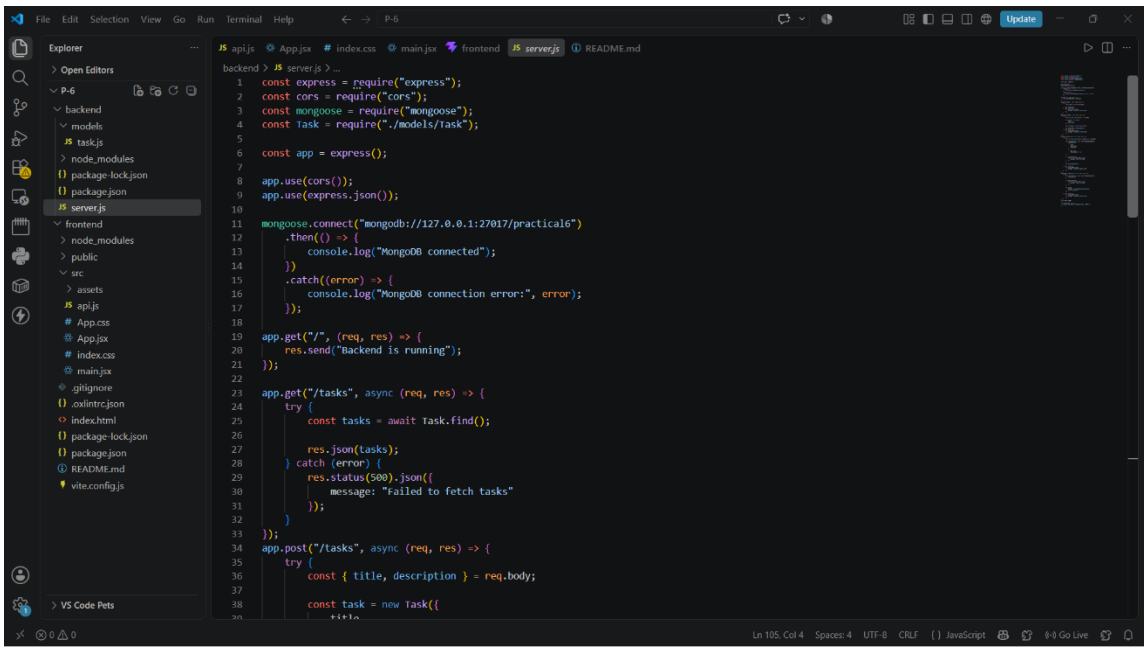
Develop a full-stack Task Manager application using React for the frontend, Node.js with Express for the backend, and MongoDB with Mongoose for data persistence. Implement RESTful APIs to perform CRUD operations (Create, Read, Update, Delete) on tasks. Integrate the React frontend with the Express backend using Fetch API, enable CORS, handle loading and error states, synchronize UI state after CRUD operations, and ensure that task data persists in MongoDB after page refresh.

Screenshots:

Task:



Server



```
const express = require("express");
const cors = require("cors");
const mongoose = require("mongoose");
const Task = require("./models/task");

const app = express();

app.use(cors());
app.use(express.json());

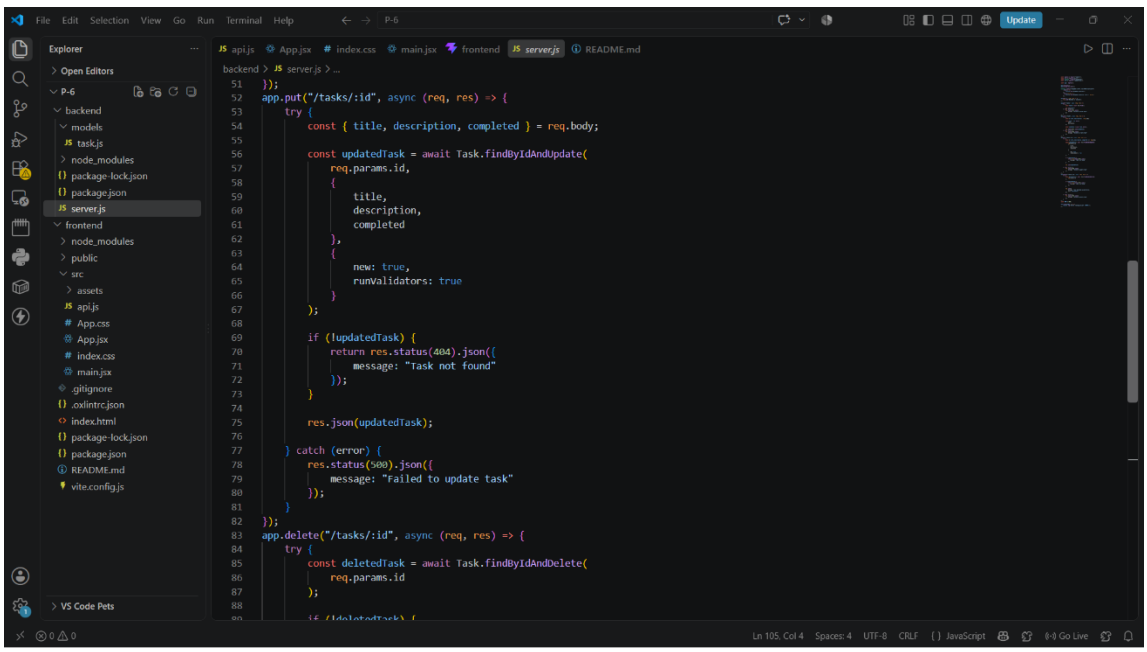
mongoose.connect("mongodb://127.0.0.1:27017/practical6")
    .then(() => {
        console.log("MongoDB connected");
    })
    .catch((error) => {
        console.log("MongoDB connection error:", error);
    });

app.get("/", (req, res) => {
    res.send("Backend is running");
});

app.get("/tasks", async (req, res) => {
    try {
        const tasks = await Task.find();

        res.json(tasks);
    } catch (error) {
        res.status(500).json({
            message: "Failed to fetch tasks"
        });
    }
});
app.post("/tasks", async (req, res) => {
    try {
        const { title, description } = req.body;

        const task = new Task({
```



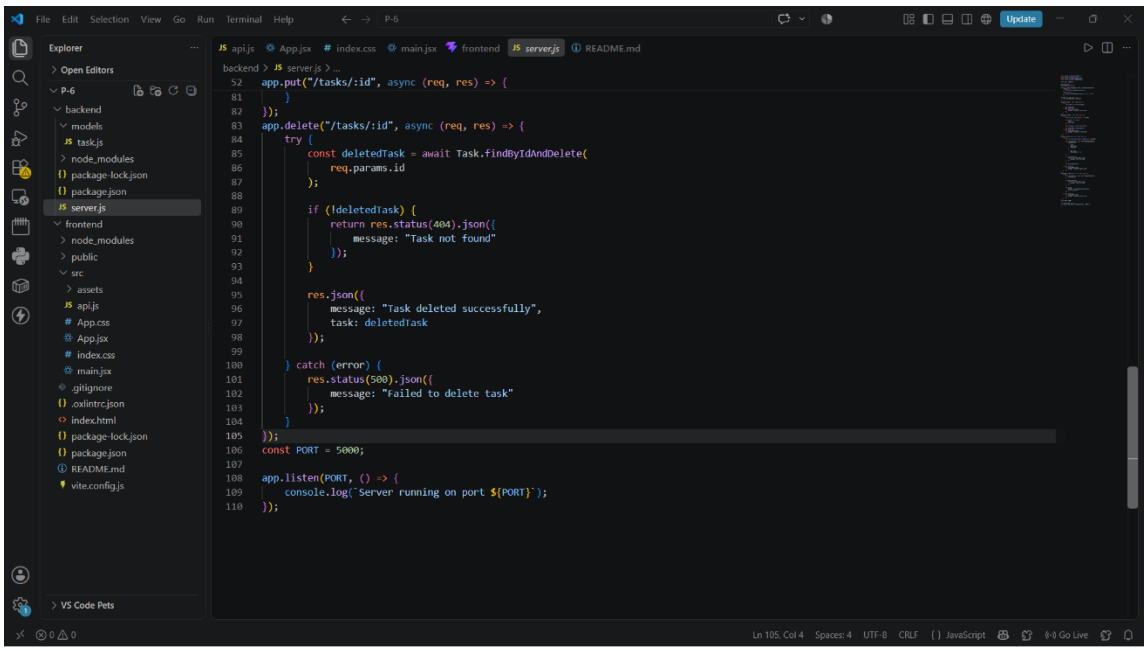
```
});
app.put("/tasks/:id", async (req, res) => {
    try {
        const { title, description, completed } = req.body;

        const updatedTask = await Task.findByIdAndUpdate(
            req.params.id,
            {
                title,
                description,
                completed
            },
            {
                new: true,
                runValidators: true
            }
        );

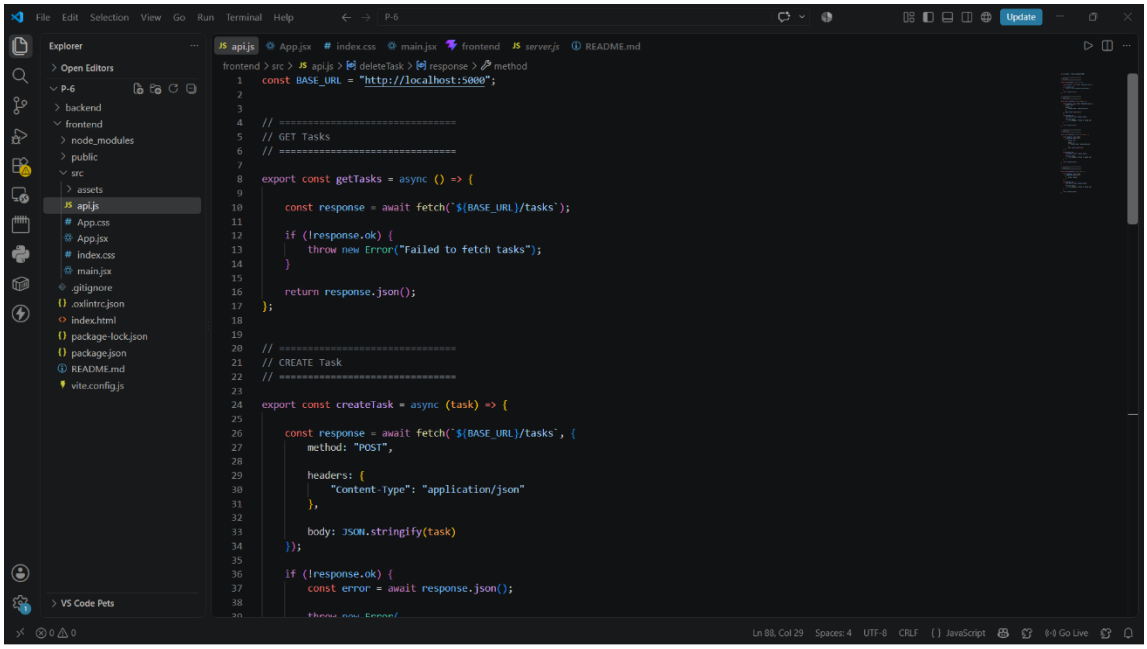
        if (!updatedTask) {
            return res.status(404).json({
                message: "Task not found"
            });
        }

        res.json(updatedTask);

    } catch (error) {
        res.status(500).json({
            message: "Failed to update task"
        });
    }
});
app.delete("/tasks/:id", async (req, res) => {
    try {
        const deletedTask = await Task.findByIdAndDelete(
            req.params.id
        );
```


File Edit Selection View Go Run Terminal Help
Update
Explorer
Open Editors
P-6
backend
models
task.js
node_modules
package-lock.json
package.json
server.js
frontend
node_modules
public
src
assets
api.js
App.css
App.jsx
index.css
main.jsx
.gitignore
.oxlintrc.json
index.html
package-lock.json
package.json
README.md
vite.config.js
VS Code Pets
api.js App.jsx index.css main.jsx frontend server.js README.md
backend > server.js > ...
app.put("/tasks/:id", async (req, res) => {
}
});
app.delete("/tasks/:id", async (req, res) => {
try {
const deletedTask = await Task.findByIdAndDelete(
req.params.id
);
if (!deletedTask) {
return res.status(404).json({
message: "Task not found"
});
}
res.json({
message: "Task deleted successfully",
task: deletedTask
});
} catch (error) {
res.status(500).json({
message: "Failed to delete task"
});
}
});
const PORT = 5000;
app.listen(PORT, () => {
console.log(`Server running on port ${PORT}`);
});
Ln 105, Col 4 Spaces: 4 UTF-8 CRLF { } JavaScript Go Live

Api:



```
// ==============================
// GET Tasks
// ==============================

export const getTasks = async () => {

    const response = await fetch(`${BASE_URL}/tasks`);

    if (!response.ok) {
        throw new Error("Failed to fetch tasks");
    }

    return response.json();
};


// ==============================
// CREATE Task
// ==============================

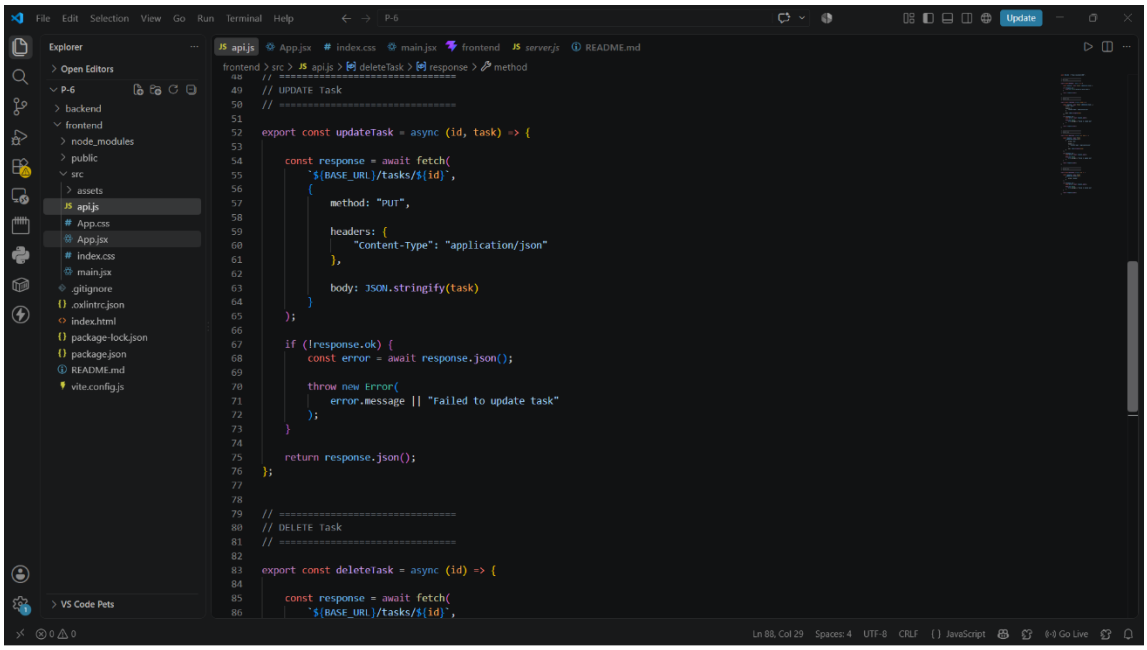
export const createTask = async (task) => {

    const response = await fetch(`${BASE_URL}/tasks`, {
        method: "POST",

        headers: {
            "Content-Type": "application/json"
        },

        body: JSON.stringify(task)
    });

    if (!response.ok) {
        const error = await response.json();
```



```
// ==============================
// UPDATE Task
// ==============================

export const updateTask = async (id, task) => {

    const response = await fetch(
        `${BASE_URL}/tasks/${id}`,
        {
            method: "PUT",

            headers: {
                "Content-Type": "application/json"
            },

            body: JSON.stringify(task)
        }
    );

    if (!response.ok) {
        const error = await response.json();

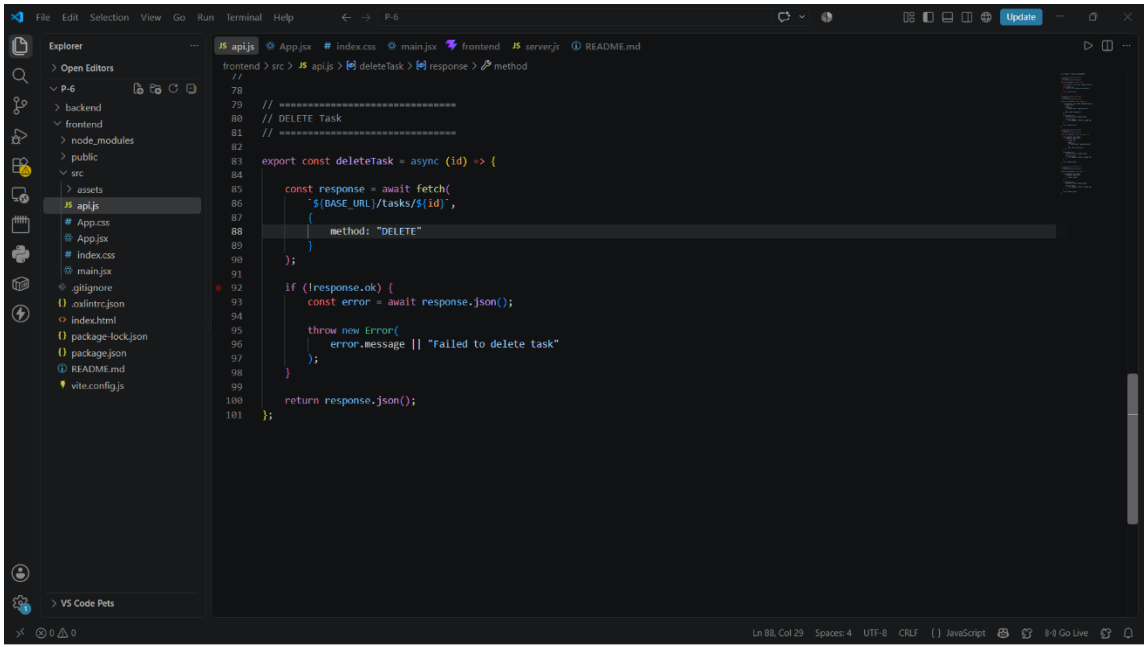
        throw new Error(
            error.message || "Failed to update task"
        );
    }

    return response.json();
};


// ==============================
// DELETE Task
// ==============================

export const deleteTask = async (id) => {

    const response = await fetch(
        `${BASE_URL}/tasks/${id}`,
```



```
// ================================
// DELETE Task
// ================================

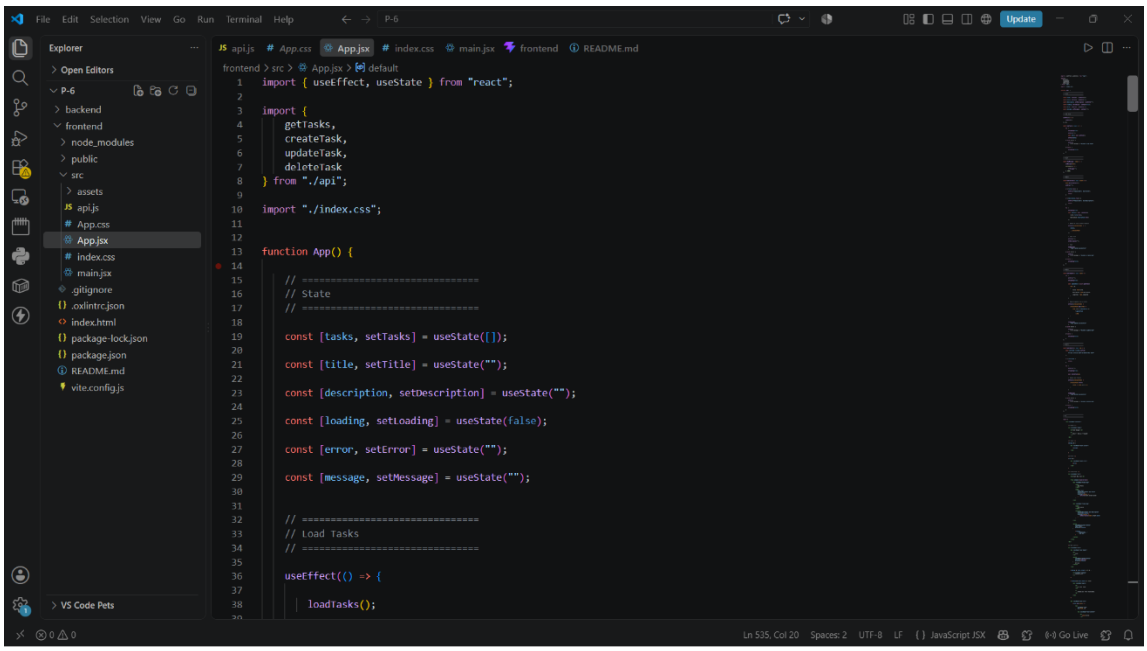
export const deleteTask = async (id) => {

    const response = await fetch(
        `${BASE_URL}/tasks/${id}`,
        {
            method: "DELETE"
        }
    );

    if (!response.ok) {
        const error = await response.json();

        throw new Error(
            error.message || "Failed to delete task"
        );
    }

    return response.json();
};
```



```
import { useEffect, useState } from "react";

import {
    getTasks,
    createTask,
    updateTask,
    deleteTask
} from "./api";

import "./index.css";

function App() {

    // ================================
    // State
    // ================================

    const [tasks, setTasks] = useState([]);

    const [title, setTitle] = useState("");

    const [description, setDescription] = useState("");

    const [loading, setLoading] = useState(false);

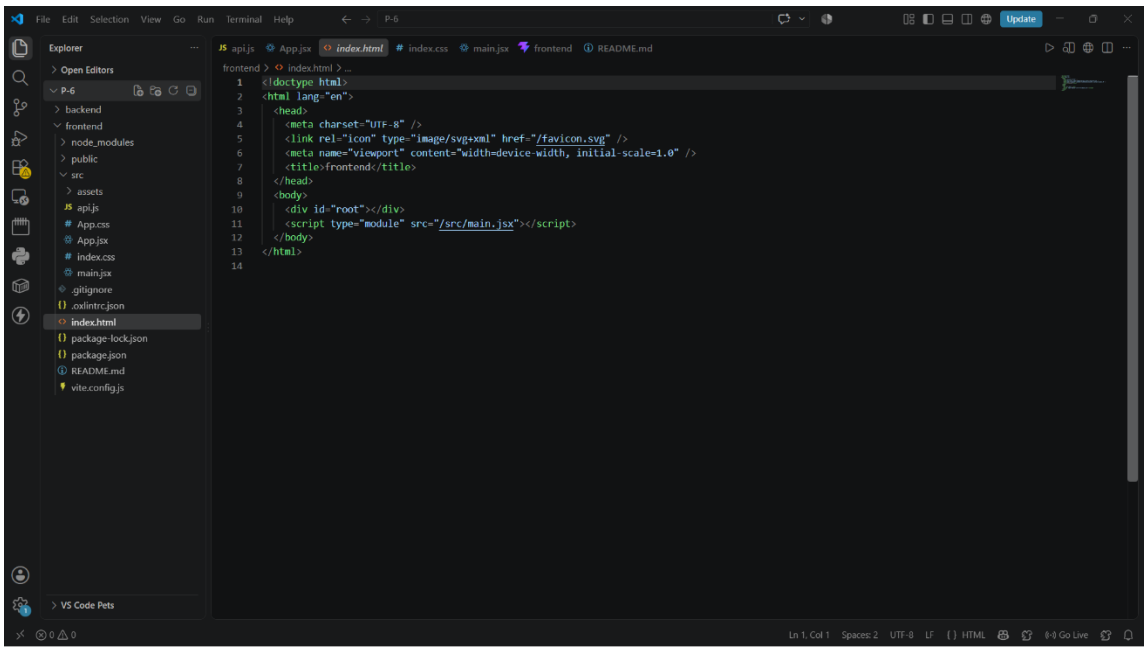
    const [error, setError] = useState("");

    const [message, setMessage] = useState("");


    // ================================
    // Load Tasks
    // ================================

    useEffect(() => {

        loadTasks();
```

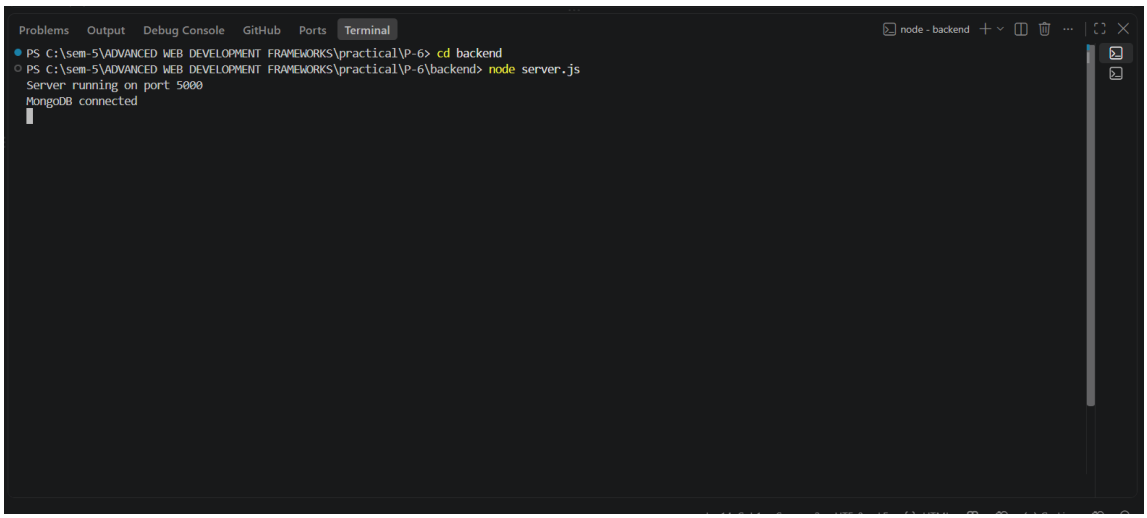

File Edit Selection View Go Run Terminal Help
P-6
Update
Explorer
Open Editors
P-6
backend
frontend
node_modules
public
src
assets
api.js
App.css
App.jsx
index.css
main.jsx
.gitignore
.oxlintrc.json
index.html
package-lock.json
package.json
README.md
vite.config.js
VS Code Pets
api.js
App.jsx
index.html
index.css
main.jsx
frontend
README.md
frontend > index.html > ...
<!doctype html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<link rel="icon" type="image/svg+xml" href="/favicon.svg" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>frontend</title>
</head>
<body>

<script type="module" src="/src/main.jsx"></script>
</body>
</html>
Ln 1, Col 1
Spaces: 2
UTF-8
LF
{ } HTML
Go Live

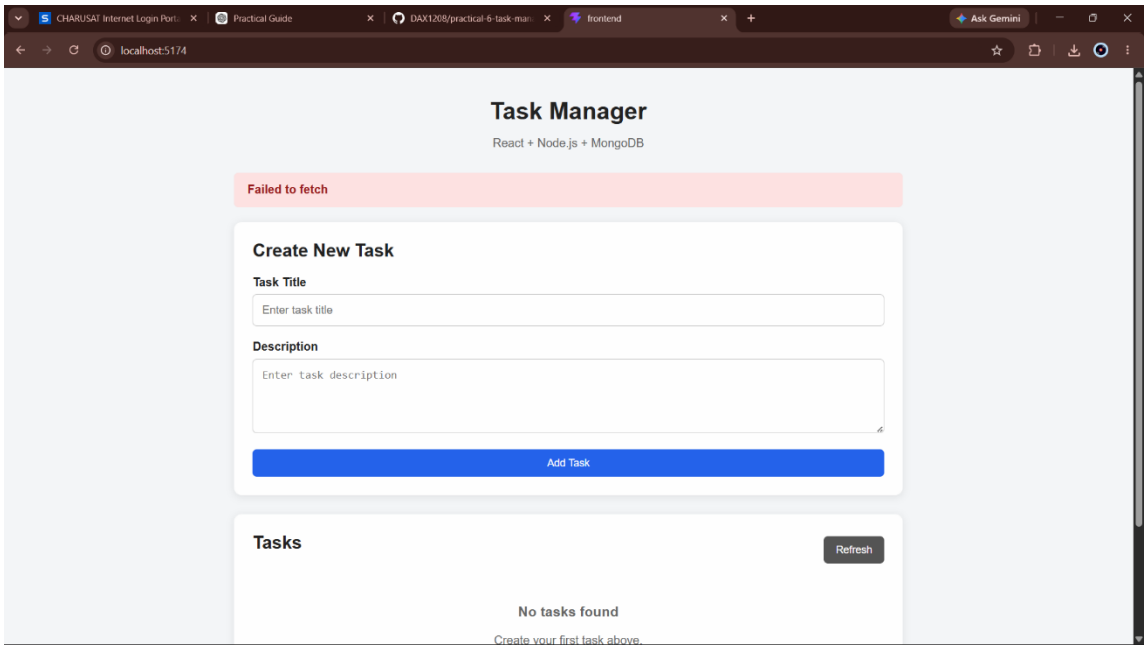
Frontend :

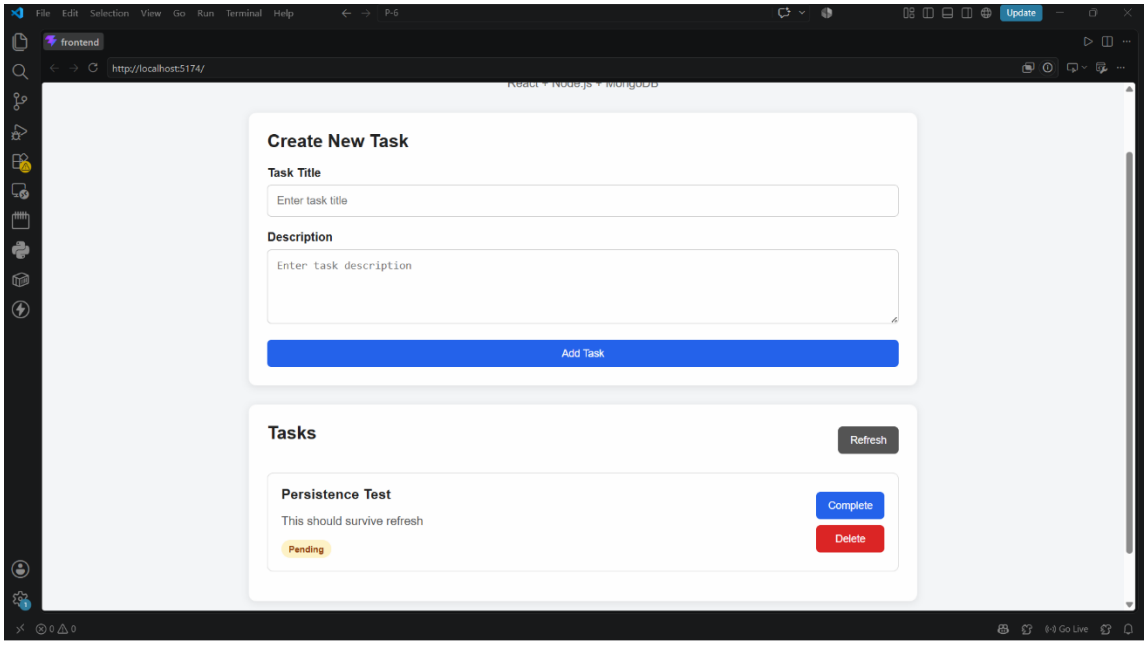

Problems Output Debug Console GitHub Ports Terminal
node - frontend
PS C:\sem-5\ADVANCED WEB DEVELOPMENT FRAMEWORKS\practical\P-6> cd frontend
PS C:\sem-5\ADVANCED WEB DEVELOPMENT FRAMEWORKS\practical\P-6\frontend> npm run dev
> frontend@0.0.0 dev
> vite
Port 5173 is in use, trying another one...
VITE v8.2.2 ready in 484 ms
→ Local: http://localhost:5174/
→ Network: use --host to expose
→ press h + enter to show help
Ln 14, Col 1 Spaces: 2 UTF-8 LF { } HTML Go Live

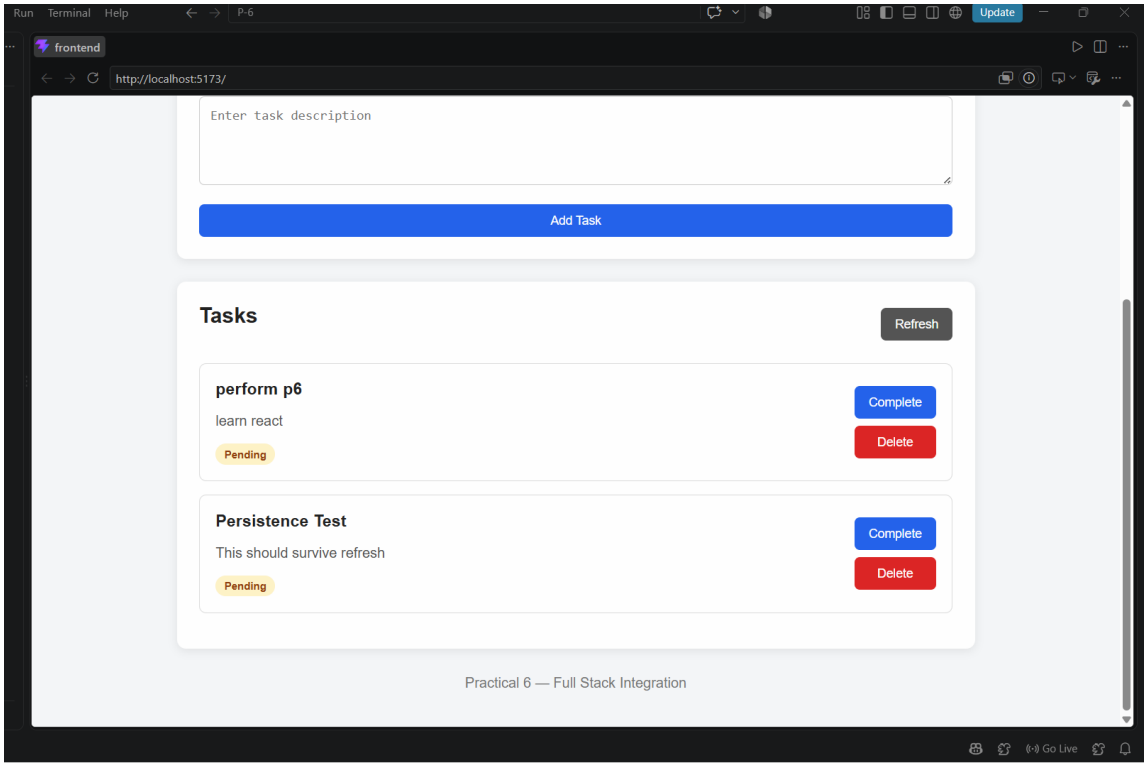
Backend:

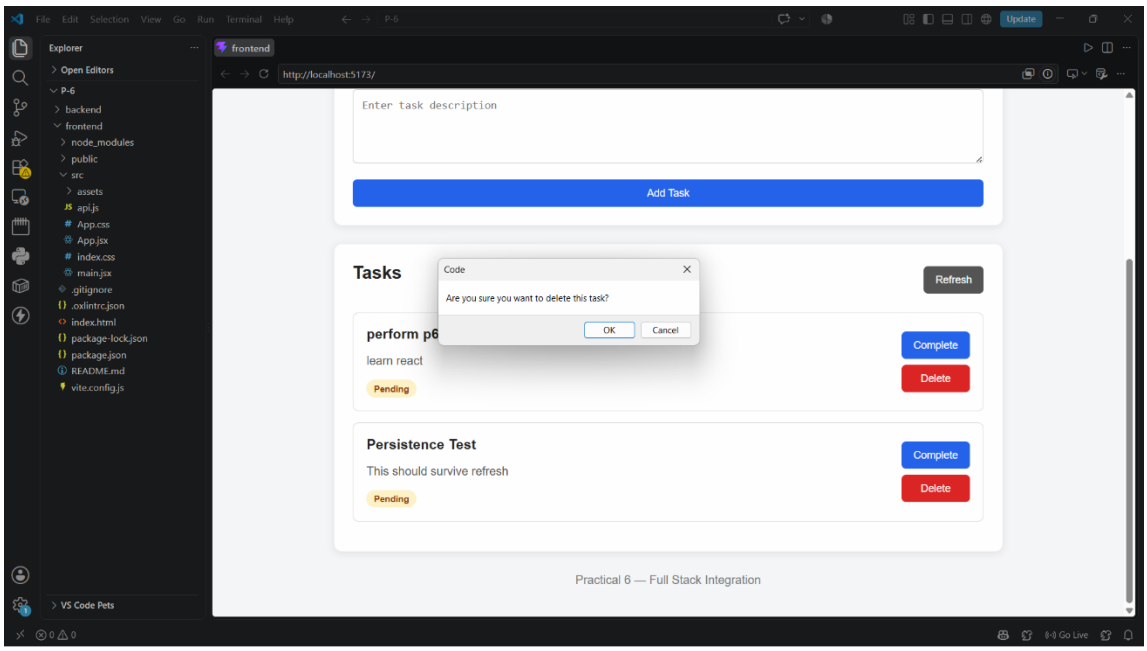

Problems Output Debug Console GitHub Ports Terminal
node - backend
PS C:\sem-5\ADVANCED WEB DEVELOPMENT FRAMEWORKS\practical\P-6> cd backend
PS C:\sem-5\ADVANCED WEB DEVELOPMENT FRAMEWORKS\practical\P-6\backend> node server.js
Server running on port 5000
MongoDB connected

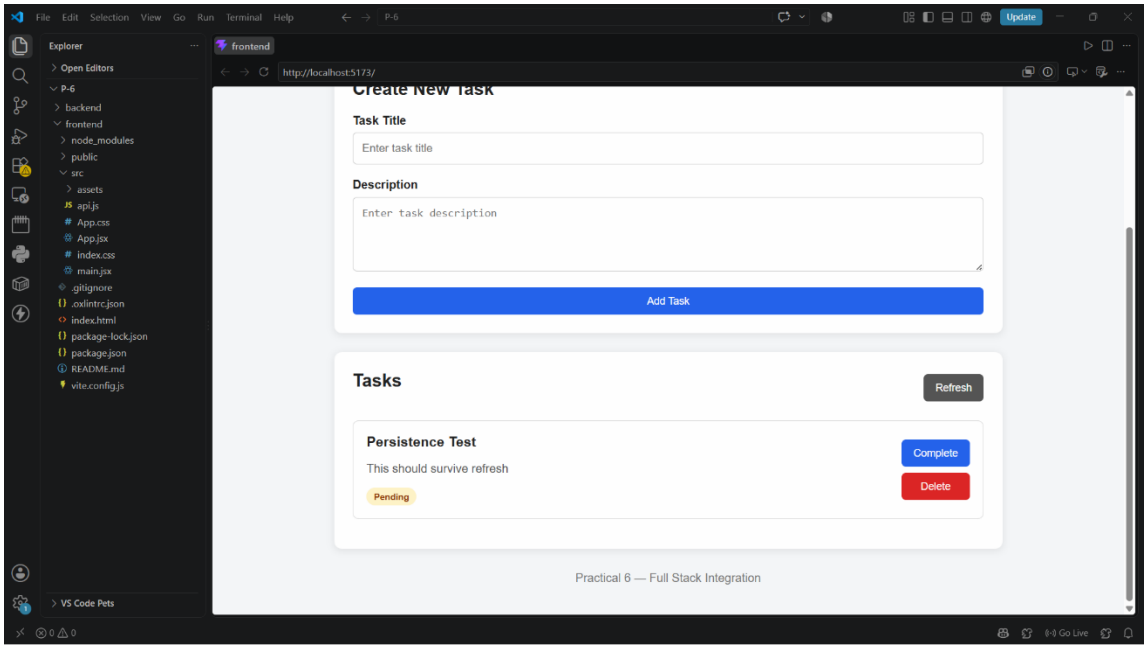
Output:

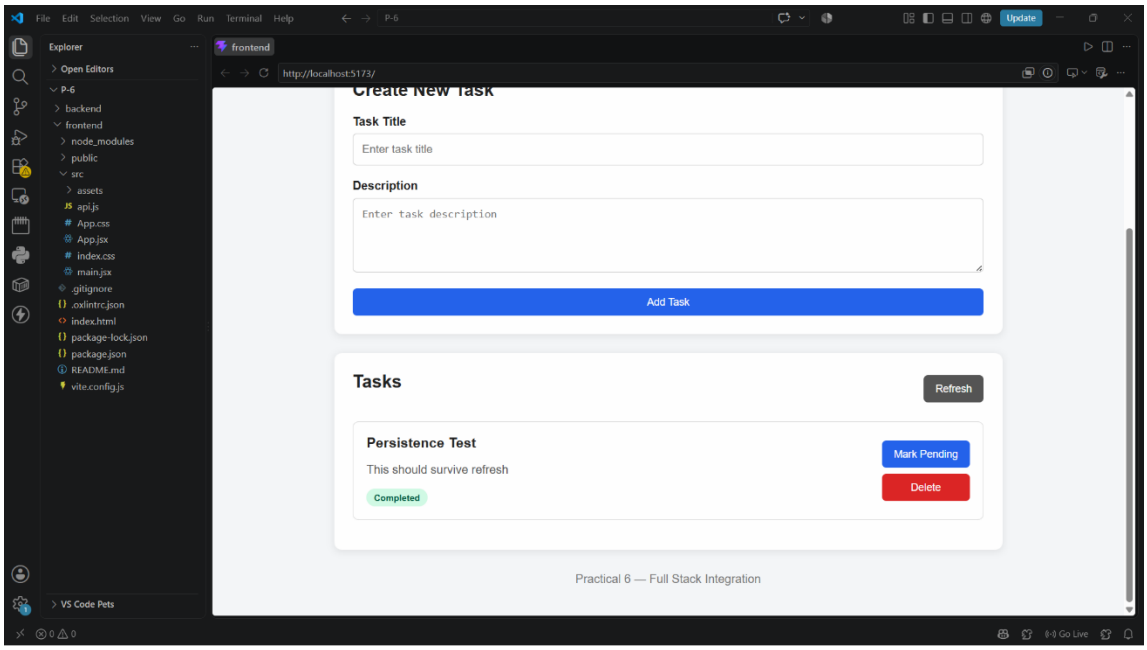

CHARUSAT Internet Login Port
Practical Guide
DAX1208/practical-6-task-man
frontend
Ask Gemini
localhost:5174
Task Manager
React + Node.js + MongoDB
Failed to fetch
Create New Task
Task Title
Enter task title
Description
Enter task description
Add Task
Tasks
Refresh
No tasks found
Create your first task above.


File Edit Selection View Go Run Terminal Help
Update
frontend
http://localhost:5174/
Create New Task
Task Title
Enter task title
Description
Enter task description
Add Task
Tasks
Refresh
Persistence Test
This should survive refresh
Pending
Complete
Delete
Go Live


Run Terminal Help
P-6
Update
frontend
http://localhost:5173/
Enter task description
Add Task
Tasks
Refresh
perform p6
learn react
Pending
Complete
Delete
Persistence Test
This should survive refresh
Pending
Complete
Delete
Practical 6 — Full Stack Integration
Go Live


File Edit Selection View Go Run Terminal Help
P-6
Update
Explorer
Open Editors
P-6
backend
frontend
node_modules
public
src
assets
api.js
App.css
App.jsx
index.css
main.jsx
.gitignore
.oxlintrc.json
index.html
package-lock.json
package.json
README.md
vite.config.js
VS Code Pets
frontend
http://localhost:5173/
Enter task description
Add Task
Tasks
Code
Are you sure you want to delete this task?
OK
Cancel
Refresh
perform p6
learn react
Pending
Complete
Delete
Persistence Test
This should survive refresh
Pending
Complete
Delete
Practical 6 — Full Stack Integration
Go Live





Github repo: https://github.com/sahilmak1602/WDF_24IT045_SAHIL.git